Thankfully, these annoyances are minor compared to the biggest issue with eval: security. This particular problem calls for a more detailed explanation.

Code Injection

Assume that, like most people, you have trouble remembering what each of the umpteen methods of Array does. As a speedy way to refresh your memory, you can write an eval-based utility that allows you to call a method on a sample array and view the result (call it the *array explorer*):

ctwc/array_explorer.rb

```
def explore_array(method)
  code = "['a', 'b', 'c'].#{method}"
  puts "Evaluating: #{code}"
  eval code
end

loop { p explore_array(gets()) }
```

The infinite loop on the last line collects strings from the standard input and feeds these strings to explore_array. In turn, explore_array turns the strings into method calls on a sample array. For example, if you feed the string *"revert()"* to explore_array, the method will evaluate the string *"['a', 'b', 'c'].revert()"*. It's time to try out this utility:

```
⇛ find_index("b")
❮ Evaluating: ['a', 'b', 'c'].find_index("b")
  1
⇛ map! {|e| e.next }
❮ Evaluating: ['a', 'b', 'c'].map! {|e| e.next }
  ["b", "c", "d"]
```

Now imagine that, being a sharing kind of person, you decide to make this program widely available on the web. You hack together a quick web page, and—presto!—you have a site where people can call array methods and see the results. (To sound like a proper startup, you might call this site "Arry" or maybe "MeThood.") Your wonderful site takes the Internet by storm, until a sneaky user feeds it a string like this:

```
⇛ object_id; Dir.glob("*")
❮ ['a', 'b', 'c'].object_id; Dir.glob("*") => [your own private information here]
```

The input is an inconsequential call to the array, followed by a command that lists all the files in your program's directory. Oh, the horror! Your malicious user can now execute arbitrary code on your computer—code that does something terrible like wipe your hard disk clean or post your love letters to your entire address book. This kind of exploit is called a *code injection* attack.